Assignment-6.3

Name: D.Suraj
2303A53006
Batch:46

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Lab Objectives
• To explore AI-powered auto-completion features for core Python constructs such as classes,
loops, and conditional statements.
• To analyze how AI tools suggest logic for object-oriented programming and control structures.
• To evaluate the correctness, readability, and completeness of AI-generated Python code.

Lab Outcomes (LOs)
After completing this lab, students will be able to:
• Use AI tools to generate and complete Python class definitions and methods.
• Understand and assess AI-suggested loop constructs for iterative tasks.
• Generate and evaluate conditional statements using AI-driven prompts.
• Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class)
Scenario
You are developing a simple student information management module.
Task
• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
• The class should include attributes such as name, roll number, and branch.
• Add a method display_details() to print student information.
• Execute the code and verify the output.
• Analyze the code generated by the AI tool for correctness and clarity.
Expected Output #1
• A Python class with a constructor (__init__) and a display_details() method.
• Sample object creation and output displayed on the console.
• Brief analysis of AI-generated code

Prompt:
Generate a Python Student class with attributes name, roll number, and branch. Include a method to display student details. Also show sample object creation and output**.**

Code:

```python
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")

# Sample object creation and output
student1 = Student("Alice Johnson", 101, "Computer Science")
student1.display_details()
print()
student2 = Student("Bob Smith", 102, "Electronics")
student2.display_details()
```

```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60163' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
Name: Alice Johnson
Roll Number: 101
Branch: Computer Science

Name: Bob Smith
Roll Number: 102
Branch: Electronics
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis:
The AI-generated class is correct and readable. It properly uses a constructor (__init__) to initialize attributes and a method to display details. Variable names are clear and follow Python conventions.

Task Description #2: Loops (Multiples of a Number)
Scenario
You are writing a utility function to display multiples of a given number.
Task
• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number
using a loop.
• Analyze the generated loop logic.
• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,
while instead of for).
Expected Output #2
• Correct loop-based Python implementation.
• Output showing the first 10 multiples of a number.
• Comparison and analysis of different looping approaches

Prompt:
Write a Python function to print the first 10 multiples of a given number using a for loop.

Code:
Using for Loop:

```python
def print_multiples(n):
    """Print the first 10 multiples of a given number."""
    for i in range(1, 11):
        print(n * i)

# Example usage
print_multiples(5)
```

```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64682' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
5
10
15
20
25
30
35
40
45
50
PS C:\Users\dussa\OneDrive\Desktop>
```

Code:

Using while Loop:

```python
def print_multiples(n):
    """Print the first 10 multiples of a given number."""
    i = 1
    while i <= 10:
        print(n * i)
        i += 1

# Example usage
print_multiples(5)
```

```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60381' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
5
10
15
20
25
30
35
40
45
50
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis

Both loops produce the same output. The for loop is more concise and readable, while the while loop provides more control over iteration.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups
(e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.

• Ask the AI to generate the same classification using alternative conditional structures (e.g.,
simplified conditions or dictionary-based logic).
Expected Output #3
• A Python function that classifies age into appropriate groups.
• Clear and correct conditional logic.
• Explanation of how the conditions work.

Prompt:
Generate a Python function using if-elif-else to classify age as child, teenager, adult, or senior.

Using if-elif-else
Code:

ai.py

C: > Users > dussa > OneDrive > Desktop > ai.py > ...

```python
def classify_age(age):
    if age < 13:
        return "child"
    elif age < 18:
        return "teenager"
    elif age < 65:
        return "adult"
    else:
        return "senior"
print(classify_age(18))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console

```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\
5.18.0-win32-x64\bundled\libs\debugpy\launcher' '49826' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
adult
PS C:\Users\dussa\OneDrive\Desktop>
```

Alternative (Dictionary-Based Logic)

ai.py

C: > Users > dussa > OneDrive > Desktop > ai.py > classify_age

```python
def classify_age(age):
    """
    Classify age into categories: child, teenager, adult, or senior.
    Args:
        age (int): The age to classify
    Returns:
        str: Age category
    """
    age_categories = {
        range(0, 13): "child",
        range(13, 20): "teenager",
        range(20, 65): "adult",
        range(65, 150): "senior"
    }
    for age_range, category in age_categories.items():
        if age in age_range:
            return category
    return "invalid age"
# Example usage
print(classify_age(10))   # child
print(classify_age(15))   # teenager
print(classify_age(30))   # adult
print(classify_age(70))   # senior
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console

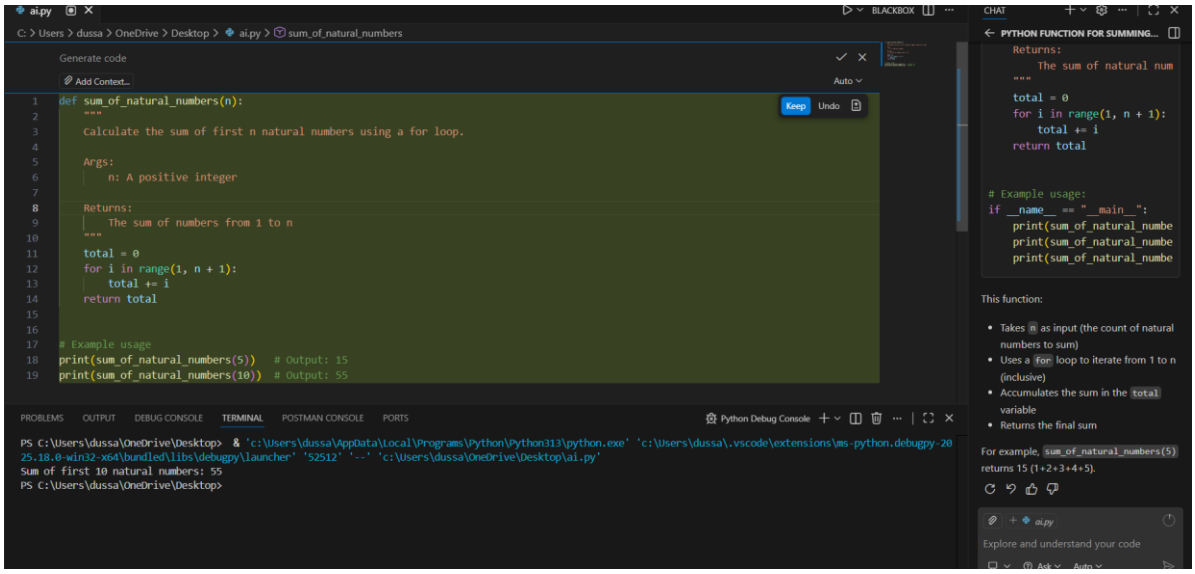
```
PS C:\Users\dussa\OneDrive\Desktop>  & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
py\launcher' '62488' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
child
teenager
adult
senior
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis
The if-elif-else structure is simpler and more readable. The dictionary-based approach is creative but slightly less intuitive for beginners.

Task Description #4: For and While Loops (Sum of First n Numbers)
Scenario
You need to calculate the sum of the first n natural numbers.
Task
• Use AI assistance to generate a sum_to_n() function using a for loop.
• Analyze the generated code.
• Ask the AI to suggest an alternative implementation using a while loop or a mathematical
formula.
Expected Output #4
• Python function to compute the sum of first n numbers.
• Correct output for sample inputs.
• Explanation and comparison of different approaches.

Prompt:
Write a Python function to calculate the sum of first n natural numbers using a for loop.

Code:



Using Mathematical Formula

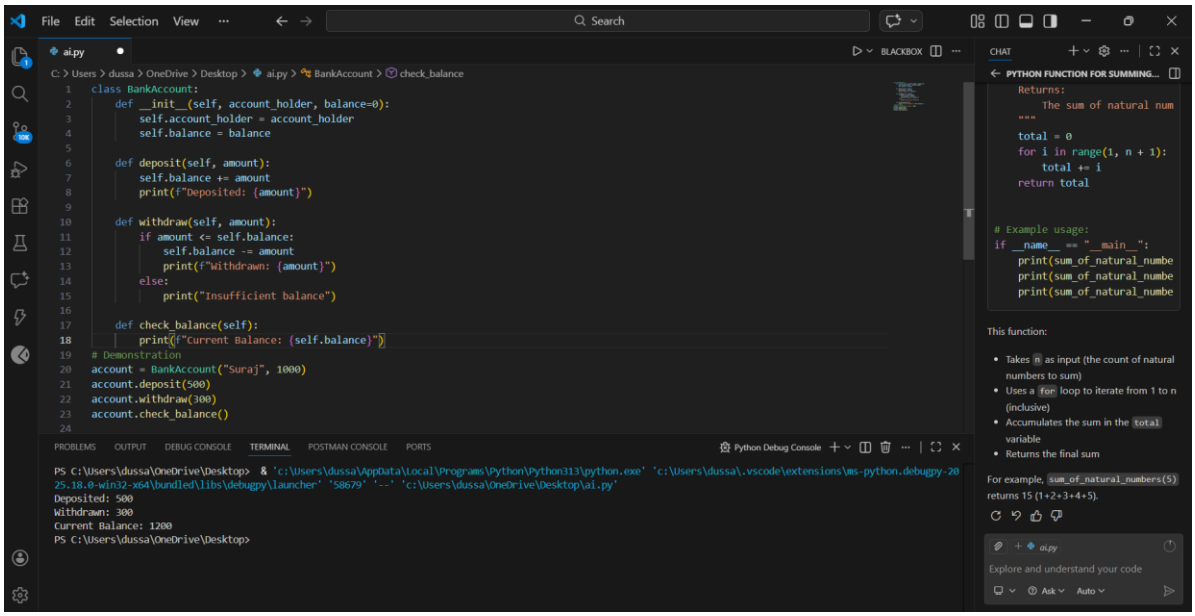
```
def sum_to_n_formula(n):
    return n * (n + 1) // 2
```

Analysis
The formula-based approach is the most efficient with O(1) time complexity. Loop-based methods are easier to understand for beginners.

Task Description #5: Classes (Bank Account Class)
Scenario
You are designing a basic banking application.
Task
• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),
and check_balance().
• Analyze the AI-generated class structure and logic.
• Add meaningful comments and explain the working of the code.
Expected Output #5
• Complete Python Bank Account class.
• Demonstration of deposit and withdrawal operations with updated balance.
• Well-commented code with a clear explanation.

Prompt:
Generate a Python BankAccount class with deposit, withdraw, and check_balance methods.

Code:
AI-Generated Code with Comments



Explanation
The class correctly encapsulates account data and operations. Conditional checks prevent invalid withdrawals. The code is clean, readable, and suitable for a basic banking application.